Galaxy Morphology MLOps Platform

Technical Design & Implementation Report

DA5402 – AI Application with MLOps

Course Assignment – Final Submission

Project name:	`galaxy-mlops-v2`
Repository:	`galaxy_morphology_fp`
Stack:	DVC · Airflow · FastAPI · Streamlit MLflow · PostgreSQL · Prometheus · Grafana
Author(s):	⟨*Add name(s)*⟩
Date:	⟨*Add date*⟩

This document covers Architecture, HLD, LLD, UML diagrams, Database schema, code walkthrough, test plan, and user manual for the Galaxy Morphology Classification Platform.

Contents

1 Project Overview **6**
1.1 Problem Statement . . . 6
1.2 Dataset . . . 6
1.3 Core Objectives Delivered . . . 7

2 Architecture **8**
2.1 High-Level Architecture Diagram . . . 8
2.2 Two-Layer Orchestration Model . . . 8
2.2.1 Layer 1 – DVC Artifact Pipeline . . . 8
2.2.2 Layer 2 – Airflow Control Plane . . . 9
2.3 Serving Architecture . . . 9

3 High-Level Design (HLD) **10**
3.1 Design Principles . . . 10
3.2 Module Map . . . 11
3.3 Data Flow Summary . . . 11
3.3.1 Training Flow . . . 11
3.3.2 Inference Flow . . . 11
3.3.3 Feedback and Continuous Improvement . . . 11
3.4 Technology Stack Justification . . . 12

4 Low-Level Design (LLD) **13**
4.1 Configuration . . . 13
4.2 DVC Pipeline Stages . . . 14
4.3 API Gateway Endpoint Specifications . . . 14
4.3.1 `GET /health` and `GET /ready` . . . 14
4.3.2 `POST /predict` . . . 15
4.3.3 `POST /predict-batch` . . . 15
4.3.4 `POST /feedback` and `POST /feedback/upload-csv` . . . 16
4.3.5 `GET /recent-predictions` . . . 16
4.4 Model Service Endpoint Specifications . . . 16
4.5 Airflow DAG Task Definitions . . . 17
4.6 ML Model Architecture . . . 17

5 UML Diagrams **18**
5.1 Component Diagram . . . 18
5.2 Sequence Diagram – Single Image Prediction . . . 19
5.3 Sequence Diagram – Airflow Retraining Loop . . . 19

6 Database Design **20**
6.1 Databases and Roles . . . 20
6.2 Application Table Definitions . . . 20
6.2.1 `prediction_batches` . . . 20
6.2.2 `predictions` . . . 21
6.2.3 `feedback_corrections` . . . 21
6.2.4 `pipeline_artifact_snapshots` . . . 21
6.3 ER Diagram . . . 22

7 Code Walkthrough **23**
7.1 Directory Structure . . . 23
7.2 Key Modules . . . 23
7.2.1 `src/common/config.py` . . . 24
7.2.2 `src/common/logging_utils.py` . . . 24
7.2.3 `src/common/artifact_store.py` . . . 24
7.2.4 `src/data/preprocess_final.py` . . . 24
7.2.5 `src/training/train.py` . . . 24
7.2.6 `src/monitoring/pipeline_exporter.py` . . . 24
7.2.7 `airflow/dags/galaxy_pipeline.py` . . . 24
7.3 Coding Standards . . . 24

8 Test Plan and Test Cases **26**
8.1 Acceptance Criteria . . . 26
8.2 Unit Test Cases . . . 26
8.3 Integration and Functional Test Cases . . . 27
8.4 Test Report Summary . . . 27
8.5 CI Pipeline . . . 27

9 MLOps Implementation Details **28**
9.1 Data Engineering . . . 28
9.2 Source Control and CI . . . 28
9.3 Experiment Tracking . . . 28
9.4 Prometheus Instrumentation . . . 29
9.5 Software Packaging and Deployment . . . 29

10 User Manual **30**
10.1 Prerequisites . . . 30
10.2 First-Time Setup . . . 30
10.3 Service URLs . . . 31
10.4 Making a Prediction . . . 31
10.5 Batch Prediction . . . 31
10.6 Submitting a Correction . . . 31
10.7 Triggering a Manual Retrain . . . 31
10.8 Viewing Monitoring Dashboards . . . 32

11 Requirement Coverage Matrix **33**

12 Deployment Runbook **35**
12.1 Environment Variables . . . 35

12.2 Starting, Verifying, and Stopping 35

A Prometheus Alert Rules **36**

B Proof of Deployment Checklist **37**

List of Figures

2.1 System architecture. Solid = request flow. Dashed = data/metrics flow. All services also expose `/metrics` to Prometheus (not all arrows shown for clarity). 8

3.1 High-Level Design module map 11

5.1 UML Component Diagram. Solid arrows = request flow; dashed = data/-metrics flow. 18
5.2 UML Sequence Diagram – Single Image Prediction 19
5.3 UML Sequence Diagram – Airflow Retraining Loop 19

6.1 Entity-Relationship diagram – `galaxy_app` database 22

List of Tables

1.1 Delivered objectives at a glance . . . 7

3.1 Technology stack and rationale . . . 12

4.1 DVC pipeline stage definitions . . . 14
4.2 `/health` and `/ready` specification . . . 14
4.3 `POST /predict` specification . . . 15
4.4 `POST /predict-batch` specification . . . 15
4.5 Feedback endpoint specifications . . . 16
4.6 `GET /recent-predictions` specification . . . 16
4.7 Model Service (`:8001`) endpoints . . . 16
4.8 Airflow control-plane tasks . . . 17

6.1 PostgreSQL databases provisioned at startup . . . 20
6.2 Table: `prediction_batches` . . . 20
6.3 Table: `predictions` . . . 21
6.4 Table: `feedback_corrections` . . . 21
6.5 Table: `pipeline_artifact_snapshots` . . . 21

8.1 Unit test cases . . . 26
8.2 Integration test cases . . . 27
8.3 Test report summary (fill after dry run) . . . 27

9.1 Prometheus metrics exposed across all services . . . 29

10.1 Service URLs after deployment . . . 31

B.1 Deployment proof screenshots (save in `image/proof/`) . . . 37

Chapter 1

Project Overview

1.1 Problem Statement

The project builds a full end-to-end MLOps platform for classifying galaxy images into five morphological categories: **Elliptical**, **Spiral**, **Lenticular**, **Irregular**, and **Merger**.

The system is designed to:

- Train and evaluate a reproducible image classifier backed by DVC.
- Expose REST prediction APIs and a user-facing Streamlit frontend.
- Capture user corrections for a continuous retraining loop.
- Store operational and application state in PostgreSQL.
- Monitor model and service behaviour with Prometheus, Grafana, Loki, and Alertmanager.
- Support full deployment via Docker Compose.

1.2 Dataset

The **Galaxy Zoo** dataset provides labelled galaxy images. Images are downloaded, validated, normalised, and split into train/val/test sets across two preprocessing stages before training.

1.3 Core Objectives Delivered

Table 1.1: Delivered objectives at a glance

Objective	**Delivery**
Reproducible ML pipeline	DVC 6-stage DAG
Operational orchestration	Airflow control-plane DAG
Application state	PostgreSQL – `galaxy_app`, `airflow`, `mlflow`
Experiment tracking	MLflow with auto-log and manual metric logging
Frontend	Streamlit – single image, batch ZIP, feedback, history
API Gateway	FastAPI `:8000`
Model Service	FastAPI `:8001`, isolated and hot-reloadable
Monitoring	Prometheus, Grafana, Loki, Promtail, Alertmanager
Continuous improvement	Feedback capture triggers retraining via Airflow
Containerisation	Docker Compose with 20+ services

Chapter 2

Architecture

2.1 High-Level Architecture Diagram

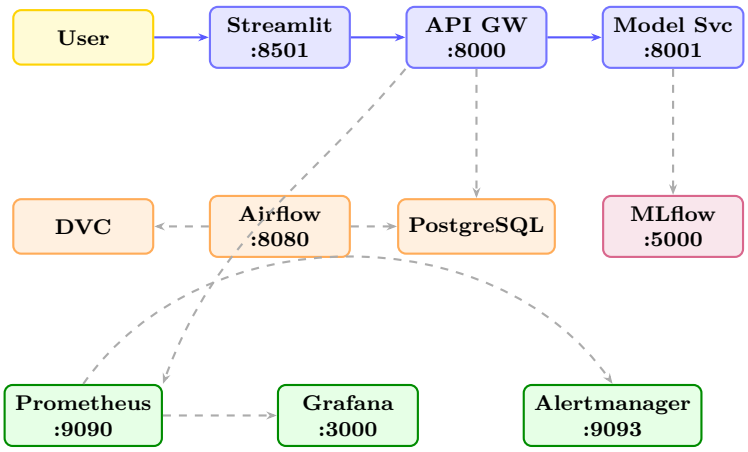

Figure 2.1: System architecture. Solid = request flow. Dashed = data/metrics flow. All services also expose `/metrics` to Prometheus (not all arrows shown for clarity).

Placeholder – add screenshot here. Replace Figure 2.1 with your actual architecture screenshot by adding:

```
\includegraphics[width=\linewidth]{image/proof/architecture.png}
```

2.2 Two-Layer Orchestration Model

2.2.1 Layer 1 – DVC Artifact Pipeline

DVC owns the deterministic artifact flow:

`fetch_raw` → `preprocess_v1` → `preprocess_final` → `train` → `evaluate` → `report`

This provides reproducibility via content-addressed artifact hashing, incremental rebuilds (only dirty stages re-run), clear DAG lineage via `dvc dag`, and CI-friendly pipeline validation.

2.2.2 Layer 2 – Airflow Control Plane

Airflow owns all runtime decisions: inspect model metrics and feedback growth, decide whether to retrain, trigger `dvc repro report`, register the best model, reload the model service, and email the generated report.

2.3 Serving Architecture

$$\textbf{Frontend} \xrightarrow{\text{REST}} \textbf{API Gateway} \xrightarrow{\text{REST}} \textbf{Model Service} \rightarrow \textbf{PostgreSQL}$$

The frontend never communicates directly with the model artifact, preserving loose coupling and clean REST boundaries.

Chapter 3

High-Level Design (HLD)

3.1 Design Principles

1. **Loose coupling:** Frontend, API gateway, and model service are independent Docker services communicating only via REST.
2. **Separation of concerns:** ML artifact pipeline (DVC) is separate from operational control (Airflow).
3. **Observability-first:** Every service exposes `/metrics` and uses structured logging.
4. **Reproducibility:** Every experiment is tied to a Git commit hash and an MLflow run ID.
5. **Fail-safe defaults:** All services expose `/health` and `/ready` for container orchestration.

3.2 Module Map

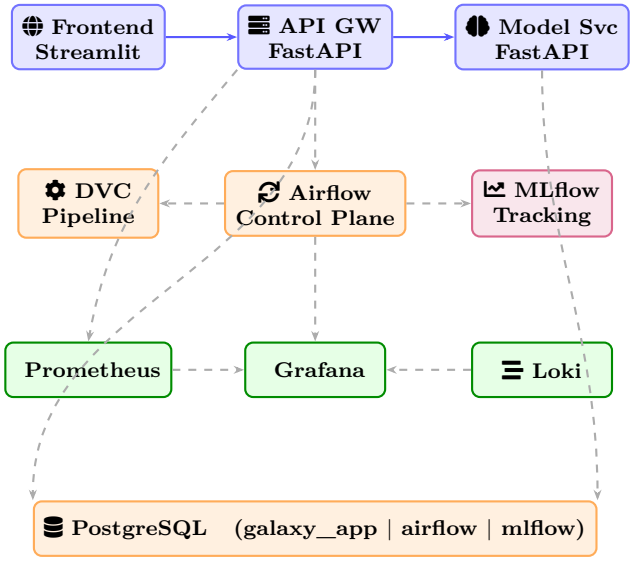

Figure 3.1: High-Level Design module map

3.3 Data Flow Summary

3.3.1 Training Flow

1. Airflow inspects model metrics and feedback count in PostgreSQL.
2. If retraining is needed it triggers `dvc repro report`.
3. DVC runs: download → preprocess → train → evaluate → report.
4. MLflow logs parameters, metrics, and model artifact.
5. The best model is registered and promoted in the MLflow model registry.
6. Airflow calls `POST /reload` on the model service, then emails the report.

3.3.2 Inference Flow

1. User uploads an image or ZIP via Streamlit.
2. Streamlit POSTs to the API gateway.
3. The gateway forwards the image to the model service.
4. ResNet-18 returns the predicted label, top-k probabilities, and brightness z-score.
5. The gateway stores the prediction in PostgreSQL and returns the result.

3.3.3 Feedback and Continuous Improvement

1. User submits a correction (prediction ID + ground-truth label) via Streamlit.

2. API gateway stores the correction in `feedback_corrections`.
3. Airflow checks feedback count against the configured threshold each cycle.
4. When exceeded, DVC retrain is triggered.
5. Feedback images are materialised into the training dataset via a CSV manifest.

3.4 Technology Stack Justification

Table 3.1: Technology stack and rationale

Tool	**Role**	**Rationale**
DVC	ML pipeline	Content-addressed artifacts; reproducibility
Airflow	Control plane	Scheduled DAG execution; conditional branching
FastAPI	API / Model svc	Async, type-safe REST; auto OpenAPI docs
Streamlit	UI	Rapid ML-friendly UI; no JavaScript required
MLflow	Experiment mgmt	Tracking, model registry, artifact store
PostgreSQL	Database	Durable relational storage for all state
Prometheus	Monitoring	Pull-based metrics scraping; alert rules
Grafana	Visualisation	Dashboard; Loki log queries
Loki	Logging	Log aggregation without full text indexing
Docker Compose	Packaging	Identical dev/prod environments; one-command deploy
ResNet-18	Model	Pretrained backbone; transfer learning

Chapter 4

Low-Level Design (LLD)

4.1 Configuration

All project behaviour is driven by `config.yaml` at the project root.

```yaml
paths:
  raw_dataset_dir: data/raw/galaxy_dataset
  processed_v1_dir: data/processed/v1
  processed_final_dir: data/processed/final
  models_dir: models/latest
  feedback_training_manifest_path: artifacts/feedback_training_manifest.csv
  drift_baseline_path: artifacts/drift_baseline.json
  latest_report_md_path: artifacts/reports/latest_report.md

data:
  image_size: 224
  normalize_mean: [0.485, 0.456, 0.406]
  normalize_std: [0.229, 0.224, 0.225]

training:
  backbone: resnet18
  batch_size: 32
  num_epochs: 10
  learning_rate: 0.001

continuous_improvement:
  min_feedback_for_retrain: 50
  min_accuracy_threshold: 0.70

services:
  api_url: http://api:8000
  model_service_url: http://model-service:8001
```

Listing 4.1: Key config.yaml structure

4.2 DVC Pipeline Stages

Table 4.1: DVC pipeline stage definitions

Stage	Module	Description
`fetch_raw`	`src.data.download_galaxy_zoo`	Downloads Galaxy Zoo dataset; writes raw images
`preprocess_v1`	`src.data.preprocess_v1`	Normalises images; validates schema
`preprocess_final`	`src.data.preprocess_final`	Train/val/test split; drift baseline
`train`	`src.training.train`	Fine-tunes ResNet-18; logs to MLflow
`evaluate`	`src.training.evaluate`	Offline test + live feedback metrics
`report`	`src.reporting.generate_report`	Markdown and HTML report

4.3 API Gateway Endpoint Specifications

4.3.1 `GET /health` and `GET /ready`

Table 4.2: `/health` and `/ready` specification

Field	/health	/ready
Method	GET	GET
Auth	None	None
Response 200	`{status:"ok", service:"api-gateway"}`	`{status:"ready"}`
Response 503	—	Model or DB not ready
Side effects	None	Sets `API_READY` and `DB_READY` gauges

4.3.2 POST /predict

Table 4.3: POST /predict specification

Field	Value
Method	POST
Content-Type	multipart/form-data
Input	file – galaxy image (PNG / JPG / BMP / WEBP)
Max size	Configurable (default 10 MB)
Response 200	PredictionResponse (schema below)
Response 400	Missing filename
Response 413	File too large
Response 500	Inference failure
Side effects	Inserts batch and prediction rows into PostgreSQL; increments API_REQUESTS and PREDICTION_ROWS_STORED

```python
class PredictionResponse(BaseModel):
    prediction_id: str
    batch_id: str | None = None
    original_filename: str | None = None
    predicted_label: str # e.g. "spiral"
    top_k: list[dict] # [{"label": "spiral", "probability": 0.87}, ...]
    model_version: str # e.g. "3"
    model_alias: str | None = None
    model_source: str | None = None
    latency_ms: float # >= 0
    brightness_zscore: float | None = None
```

Listing 4.2: PredictionResponse Pydantic schema

4.3.3 POST /predict-batch

Table 4.4: POST /predict-batch specification

Field	Value
Method	POST
Content-Type	multipart/form-data
Input	zip_file – ZIP archive containing galaxy images
Max size	Configurable (default 50 MB)
Response 200	BatchPredictionResponse{batch_id, total_images, items[]}
Response 400	Not a ZIP or no supported images inside
Response 413	ZIP too large

4.3.4 POST /feedback and POST /feedback/upload-csv

Table 4.5: Feedback endpoint specifications

Field	POST /feedback	POST /feedback/upload-csv
Content-Type	`application/json`	`multipart/form-data`
Input	`{prediction_id, ground_truth_label, notes?}`	CSV with `prediction_id`, `corrected_label` columns
Response 200	`{status:"recorded"}`	`{status, row_count, upload_id}`
Response 400	Validation error	Non-CSV file or row errors
Side effects	Inserts into `feedback_corrections`; increments `FEEDBACK_COUNT`	Bulk-inserts corrections

4.3.5 GET /recent-predictions

Table 4.6: `GET /recent-predictions` specification

Field	Value
Method	GET
Query params	`limit` (1–5000, default 50), `start_date`, `end_date`
Response 200	`{items: [...], summary: {...}}`

4.4 Model Service Endpoint Specifications

Table 4.7: Model Service (`:8001`) endpoints

Endpoint	Method	Description
`/health`	GET	Returns `{status:"ok"}`
`/ready`	GET	200 if model loaded, 503 otherwise
`/reload`	POST	Hot-reloads model from `models/latest`; called by Airflow
`/predict`	POST	Image → `ModelPredictionResponse` with label, top-k, latency, brightness z-score, model version
`/metrics`	GET	Prometheus metrics endpoint

4.5 Airflow DAG Task Definitions

Table 4.8: Airflow control-plane tasks

Task ID	Operator	Logic
`inspect_and_branch`	BranchPython	Reads metrics and feedback count; branches to retrain or skip
`run_dvc_pipeline`	Python	Runs `dvc repro report` in subprocess
`register_best_model`	Python	Queries MLflow for best run; promotes in model registry
`reload_model_service`	Python	POSTs to `/reload` on model service
`refresh_report_after_registry`	Python	Regenerates report after registry update
`send_report_email`	Python	Reads HTML report; sends via Airflow SMTP hook

4.6 ML Model Architecture

Model: ResNet-18 (torchvision, pretrained on ImageNet)
Modification: Final FC layer replaced – `nn.Linear(512, num_classes)`
Classes: 5 – elliptical, spiral, lenticular, irregular, merger
Input: RGB images resized to 224×224, normalised with ImageNet statistics
Loss: CrossEntropyLoss **Optimiser:** Adam (LR from config, default 0.001)
Feedback loop: Feedback images combined via `ConcatDataset` and `FeedbackManifestDataset`

Chapter 5

UML Diagrams

5.1 Component Diagram

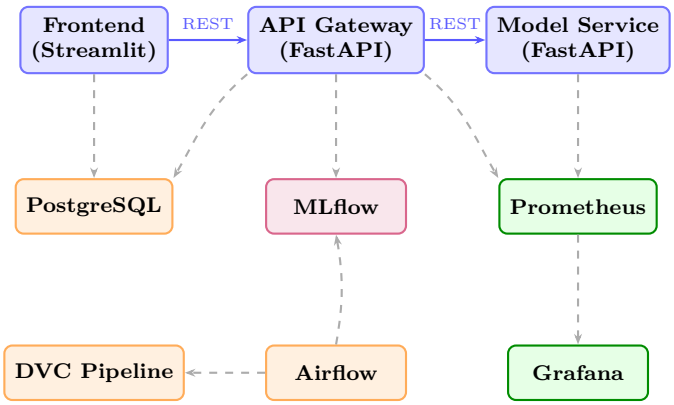

Figure 5.1: UML Component Diagram. Solid arrows = request flow; dashed = data/metrics flow.

Placeholder – insert screenshot here.

`\includegraphics[width=\linewidth]{image/proof/uml_component.png}`

5.2 Sequence Diagram – Single Image Prediction

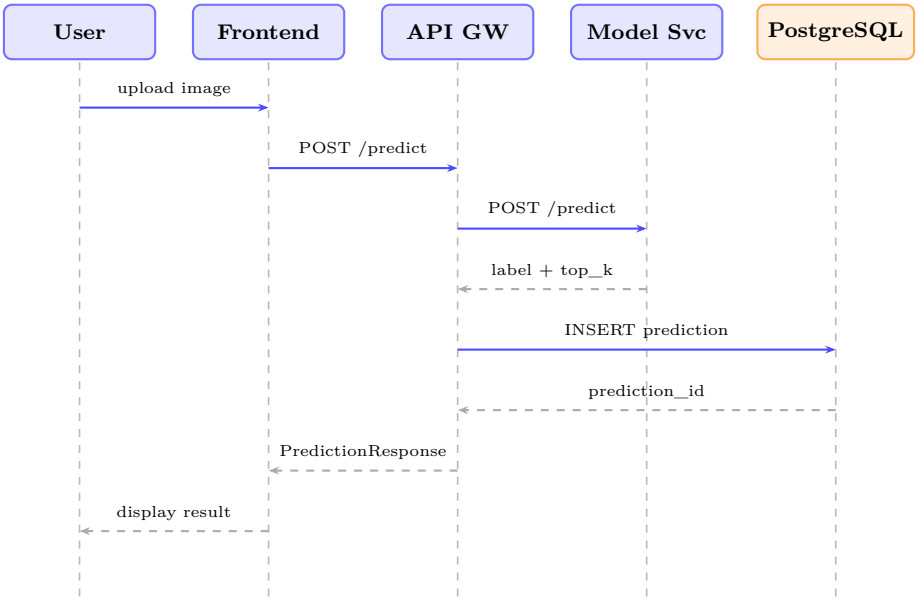

Figure 5.2: UML Sequence Diagram – Single Image Prediction

5.3 Sequence Diagram – Airflow Retraining Loop

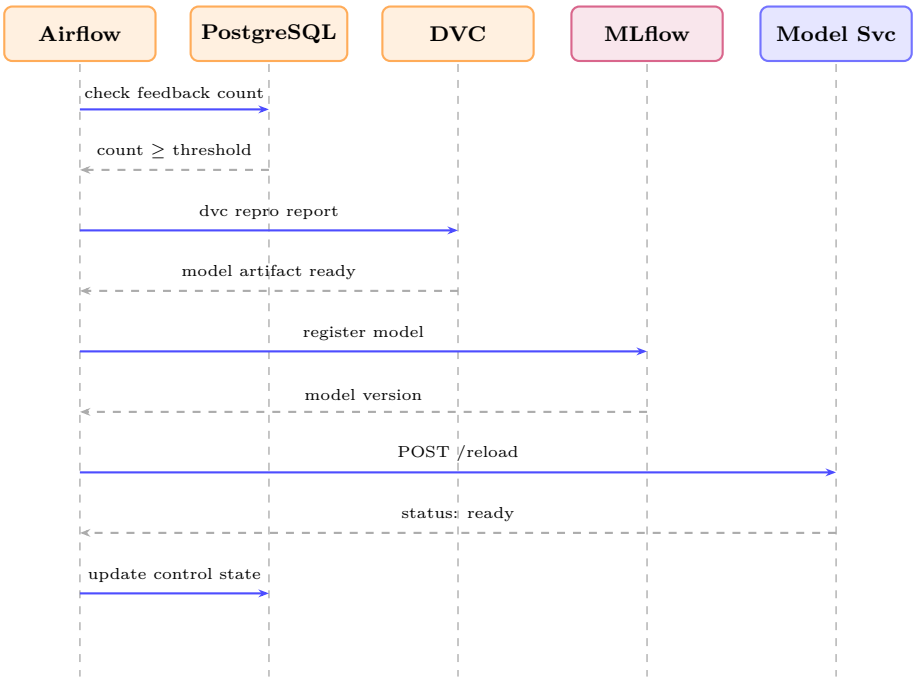

Figure 5.3: UML Sequence Diagram – Airflow Retraining Loop

Chapter 6

Database Design

6.1 Databases and Roles

Table 6.1: PostgreSQL databases provisioned at startup

Database	Owner	Purpose
`airflow`	`airflow`	Airflow metadata, DAG runs, task instances
`galaxy_app`	`galaxy`	Application tables (predictions, feedback)
`mlflow`	`mlflow`	MLflow runs, params, metrics, artifacts

6.2 Application Table Definitions

6.2.1 `prediction_batches`

Table 6.2: Table: `prediction_batches`

Column	Type	Description
`batch_id`	TEXT PK	Unique batch identifier
`source_filename`	TEXT	Name of uploaded file or ZIP
`source_type`	TEXT	`single_image` or `zip_archive`
`total_files`	INTEGER	Number of images in batch
`created_at`	TIMESTAMPTZ	Batch creation timestamp

6.2.2 predictions

Table 6.3: Table: predictions

Column	Type	Description
prediction_id	TEXT PK	Unique prediction identifier
batch_id	TEXT FK	References prediction_batches
original_filename	TEXT	Source image filename
image_data	BYTEA	Raw image bytes
predicted_label	TEXT	Model output class
top_k	JSONB	Top-k class probabilities
model_version	TEXT	MLflow model version
latency_ms	FLOAT	End-to-end inference latency
brightness_zscore	FLOAT	Drift signal
created_at	TIMESTAMPTZ	Inference timestamp

6.2.3 feedback_corrections

Table 6.4: Table: feedback_corrections

Column	Type	Description
id	SERIAL PK	Auto-increment row ID
prediction_id	TEXT FK	References predictions
predicted_label	TEXT	Original model prediction
corrected_label	TEXT	User-provided ground truth
notes	TEXT	Optional annotation
created_at	TIMESTAMPTZ	Correction timestamp

6.2.4 pipeline_artifact_snapshots

Table 6.5: Table: pipeline_artifact_snapshots

Column	Type	Description
id	SERIAL PK	Row ID
artifact_key	TEXT	e.g. train_metrics, test_metrics
stage_name	TEXT	DVC stage that produced this artifact
payload	JSONB	Metric or summary payload
recorded_at	TIMESTAMPTZ	When the snapshot was stored
recorded_date	DATE	Partition key

The view latest_pipeline_artifact_snapshots returns the most recent snapshot per artifact_key.

6.3 ER Diagram

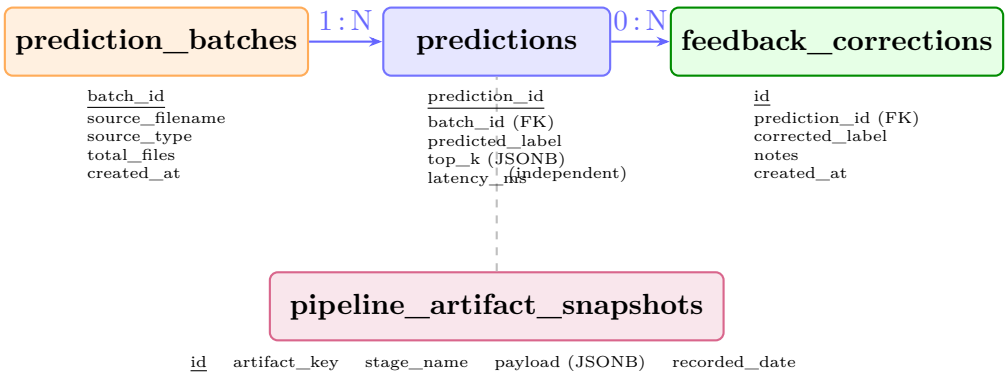

Figure 6.1: Entity-Relationship diagram – `galaxy_app` database

Placeholder – insert ER screenshot here.

`\includegraphics[width=\linewidth]{image/proof/er_diagram.png}`

Chapter 7

Code Walkthrough

7.1 Directory Structure

```
galaxy-mlops-v2/
  api/app/
    main.py # Route handlers and Prometheus metrics
    schemas.py # Pydantic I/O models
    model_client.py # HTTP client to model-service
    feedback_store.py # PostgreSQL CRUD for predictions and feedback
    metrics.py # Prometheus Counter/Gauge/Histogram definitions
  model_service/app/
    main.py # Route handlers
    model_loader.py # MLflow artifact loader and GalaxyPredictor
    schemas.py # Pydantic I/O models
  frontend/
    app.py # Main page: single/batch/feedback
    pages/1_Pipeline_Console.py # Pipeline status page
  src/
    common/ # config, logging, postgres, artifact_store
    data/ # download, ingest, preprocess_v1, preprocess_final
    features/ # drift baseline computation
    training/ # model_def, train, evaluate
    reporting/ # generate_report
    monitoring/ # pipeline_exporter Prometheus app
    registry/ # MLflow model promotion
  airflow/dags/galaxy_pipeline.py # Airflow control-plane DAG
  monitoring/ # Prometheus, Grafana, Loki, Alertmanager configs
  postgres/init/ # Database initialisation SQL scripts
  tests/unit/ # test_preprocess, test_schemas, test_metrics
  tests/integration/ # test_health_contracts
  .github/workflows/ci.yml # GitHub Actions CI
  dvc.yaml # DVC pipeline definition
  docker-compose.yml # Full 20-service Docker Compose stack
  config.yaml # Central configuration
  MLproject # MLflow project entry point
```

Listing 7.1: Top-level project structure

7.2 Key Modules

7.2.1 src/common/config.py

Central configuration loader. Reads `config.yaml` from the path set via `APP_CONFIG_PATH`, supports nested key access via dot-notation, and resolves paths relative to `PROJECT_ROOT`.

7.2.2 src/common/logging_utils.py

Provides `configure_logging(service_name)` which returns a structured logger. Records are stored in the `service_logs` PostgreSQL table in addition to stdout, enabling Loki ingestion without any additional setup.

7.2.3 src/common/artifact_store.py

Wraps DVC artifact I/O with PostgreSQL snapshotting. `store_pipeline_artifact` persists a JSON snapshot to `pipeline_artifact_snapshots`; `load_pipeline_artifact` retrieves the latest snapshot via the view.

7.2.4 src/data/preprocess_final.py

Applies train/val/test split, calls `compute_image_baseline` to calculate brightness statistics per class, and writes the drift baseline file.

7.2.5 src/training/train.py

Builds the ResNet-18 classifier, combines base training data with feedback samples via `ConcatDataset`, runs the training loop with MLflow autolog, logs confusion matrix and per-class F1, and saves the model artifact.

7.2.6 src/monitoring/pipeline_exporter.py

A lightweight FastAPI app that loads pipeline metrics from PostgreSQL artifact snapshots and exposes them as Prometheus gauges. Exports test accuracy, macro F1, live accuracy, feedback count, training duration, raw image counts by class, database row counts, and log record counts by service and level.

7.2.7 airflow/dags/galaxy_pipeline.py

Control-plane DAG: inspects PostgreSQL for control state and feedback count, branches on whether retraining is required, runs `dvc repro report` in a subprocess, queries MLflow for the best run, registers and promotes the model, POSTs to the model service reload endpoint, and emails the HTML report.

7.3 Coding Standards

- `from __future__ import annotations` in all source files (PEP 563).
- Type hints throughout (PEP 484).
- Explicit `except Exception as exc` with contextual re-raise in all handlers.

- Named loggers via `configure_logging` in every module.
- Pydantic v2 models for all API input and output.
- `compileall` pass in CI validates syntax of every `.py` file.

Chapter 8

Test Plan and Test Cases

8.1 Acceptance Criteria

Definition of Done – the system is acceptable when all of the following hold:

1. `dvc dag` renders without errors.
2. `dvc repro report` completes end-to-end.
3. All unit tests pass (`pytest tests/ -q`).
4. The frontend successfully predicts a galaxy image.
5. Feedback submission is reflected in PostgreSQL.
6. Prometheus scrapes all three targets successfully.
7. Grafana dashboard displays live metrics without empty panels.
8. Airflow DAG completes a full cycle (retrain + email).

8.2 Unit Test Cases

Table 8.1: Unit test cases

ID	File	Test	Expected
UT-01	`test_preprocess`	`split_counts(101, 0.70, 0.15)` sums to 101	PASS
UT-02	`test_preprocess`	`DatasetValidationResult` dataclass fields correct	PASS
UT-03	`test_schemas`	`FeedbackRequest` accepts valid payload	PASS
UT-04	`test_schemas`	`PredictionResponse` accepts valid payload	PASS
UT-05	`test_metrics`	Empty folder returns zero brightness baseline	PASS

8.3 Integration and Functional Test Cases

Table 8.2: Integration test cases

ID	Component	Test	Expected
IT-01	DVC	`dvc dag` renders pipeline	6 nodes
IT-02	DVC	`dvc repro report` end-to-end	All stages pass
IT-03	Airflow	Control-plane DAG triggers correctly	Correct branch taken
IT-04	API	`GET /health` returns 200	`{status:"ok"}`
IT-05	API	`GET /ready` when model loaded	200
IT-06	Frontend	Upload image; receive prediction	Valid class label
IT-07	Feedback	Submit correction; row appears in Postgres	Count ≥ 1
IT-08	Model Svc	`POST /reload` after retrain	200
IT-09	Prometheus	All 3 scrape targets show UP	Healthy
IT-10	Grafana	Dashboard panels display metrics	No empty panels
IT-11	Loki	App logs visible in Grafana Explore	Log lines returned
IT-12	Alertmanager	Alert email delivered to Mailtrap	Email received
IT-13	CI	GitHub Actions pipeline passes	All steps green

8.4 Test Report Summary

Action required: Run `pytest tests/ -v` after your final dry run and fill in the table below before submission.

Table 8.3: Test report summary (fill after dry run)

Category	Total	Passed	Failed
Unit tests	5		
Integration tests	13		
Total	18		

8.5 CI Pipeline

The `.github/workflows/ci.yml` workflow runs on every push to `main` and on all pull requests, executing five steps in order:

1. Checkout repository and set up Python 3.10.
2. Install `pytest`, `fastapi`, `pydantic`, `prometheus-client`, `pillow`, `pyyaml`, `dvc`.
3. `python -m compileall src api/app model_service/app frontend tests airflow/dags`
4. `dvc dag` – validates pipeline DAG definition.
5. `pytest tests -q` – runs all unit tests.

Chapter 9

MLOps Implementation Details

9.1 Data Engineering

- **Two-stage preprocessing:** Stage 1 (`preprocess_v1`) normalises and validates schema. Stage 2 (`preprocess_final`) performs the train/val/test split and computes the drift baseline.
- **Data validation:** `src/data/validate.py` implements schema consistency checks and per-class image counts via `DatasetValidationResult`.
- **Drift baseline:** Brightness mean and variance per class computed in `src/features/baseline.py`; stored in `artifacts/drift_baseline.json`.
- **Orchestration:** Airflow's control-plane DAG wraps the full DVC pipeline including data ingestion.

9.2 Source Control and CI

- **Git** for code versioning; **DVC** for data and model versioning.
- `dvc.lock` records exact artifact content hashes for reproducibility.
- **GitHub Actions** CI: syntax check, DAG validation, and unit tests on every push.
- Each MLflow run ID is tied to its Git commit hash, ensuring full traceability.

9.3 Experiment Tracking

- MLflow **autolog** enabled for PyTorch (captures loss curves and epoch metrics automatically).
- **Manual logging:** per-class precision, recall, and F1; confusion matrix CSV; training duration; number of feedback samples included.
- **Model registry:** Best run promoted to the `champion` alias via `src/registry/register_best_mod`
- Tracked artifacts: class names JSON, confusion matrix, model checkpoint.

9.4 Prometheus Instrumentation

Table 9.1: Prometheus metrics exposed across all services

Metric	Type	Description
`galaxy_api_requests_total`	Counter	API requests by endpoint and status
`galaxy_api_latency_seconds`	Histogram	API end-to-end latency
`galaxy_api_ready`	Gauge	API readiness (0 or 1)
`galaxy_db_ready`	Gauge	DB readiness (0 or 1)
`galaxy_model_requests_total`	Counter	Model inference calls by status
`galaxy_model_latency_seconds`	Histogram	Model inference latency
`galaxy_model_ready`	Gauge	Model loaded (0 or 1)
`galaxy_input_brightness_zscore`	Gauge	Drift signal for latest request
`galaxy_pipeline_test_accuracy`	Gauge	Latest offline test accuracy
`galaxy_pipeline_test_macro_f1`	Gauge	Latest offline macro F1
`galaxy_live_feedback_accuracy`	Gauge	Feedback loop live accuracy
`galaxy_live_feedback_count`	Gauge	Total feedback corrections
`galaxy_pipeline_train_duration_seconds`	Gauge	Last training duration
`galaxy_raw_images_total`	Gauge	Raw images by class label
`galaxy_db_prediction_count`	Gauge	Total predictions in Postgres
`galaxy_db_correction_count`	Gauge	Total corrections in Postgres
`galaxy_service_logs_total`	Gauge	Log records by service and level

9.5 Software Packaging and Deployment

- **FastAPI** exposes the inference API. **MLflow Projects** (`MLproject`) defines the training entry point and environment.
- **Docker:** Custom Dockerfiles for the API, model service, frontend, Airflow, and trainer.
- **Docker Compose:** 20+ services with health checks, dependency ordering, and volume mounts.
- **Adminer** is included for interactive SQL querying and database proof screenshots.
- **Environment parity:** All services share the `.env` file for secret injection, ensuring identical configuration across development and deployment.

Chapter 10

User Manual

10.1 Prerequisites

- Docker Desktop or Docker Engine with the Docker Compose plugin installed.
- At least 8 GB of free RAM.
- `.env` file configured (copy from `.env.example`).

10.2 First-Time Setup

```
# 1. Clone the repository
git clone <repo_url>
cd galaxy_morphology_fp

# 2. Configure environment
cp .env.example .env
# Edit .env: set AIRFLOW_UID, Mailtrap SMTP credentials, etc.

# 3. Start all services
docker compose up -d --build

# 4. Verify all containers are running
docker compose ps

# 5. Wait 2-3 minutes for Airflow to initialise
```

10.3 Service URLs

Table 10.1: Service URLs after deployment

Service	URL	
Frontend (main UI)	`http://localhost:8501`	
API (Swagger docs)	`http://localhost:8000/docs`	
Model Service docs	`http://localhost:8001/docs`	
Airflow	`http://localhost:8080`	(airflow / airflow)
MLflow	`http://localhost:5000`	
Prometheus	`http://localhost:9090`	
Alertmanager	`http://localhost:9093`	
Grafana	`http://localhost:3000`	(admin / admin)
Adminer (DB UI)	`http://localhost:8081`	

10.4 Making a Prediction

1. Open `http://localhost:8501` in a browser.
2. Click the **Single image** tab.
3. Click **Browse files** and select a galaxy image (PNG, JPG, BMP, or WEBP).
4. Click **Predict Morphology**.
5. The predicted class, top-5 probabilities, model version, and inference latency are displayed.

10.5 Batch Prediction

1. Click the **ZIP batch upload** tab.
2. Upload a ZIP file containing galaxy images.
3. Results appear in a table; export as CSV via the button provided.

10.6 Submitting a Correction

1. Click the **Correction CSV upload** tab.
2. Download the CSV template.
3. Fill in `prediction_id` and `corrected_label` for each prediction to correct.
4. Upload the completed CSV. Validation errors (if any) appear in a dialog for correction.

10.7 Triggering a Manual Retrain

1. Open Airflow at `http://localhost:8080`.

2. Locate the **galaxy_morphology_control_plane** DAG.
3. Click the **Play** button to trigger a manual run.
4. Monitor task states in the DAG grid view.

10.8 Viewing Monitoring Dashboards

1. Open Grafana at `http://localhost:3000` (admin / admin).
2. Navigate to **Dashboards** → **Galaxy MLOps Dashboard**.
3. Panels include API request rate, model latency, prediction count, feedback count, pipeline accuracy, drift z-score, and log streams.

Chapter 11

Requirement Coverage Matrix

Requirement	Status	Evidence
Frontend UI (intuitive, non-technical)	✔ Done	`frontend/app.py`; 4-tab Streamlit UI
ML Pipeline Visualisation screen	✔ Done	Pipeline Console page; Airflow + MLflow UIs
Pipeline management console	✔ Done	Airflow DAG grid view
Error/failure tracking console	✔ Done	Airflow task logs; Grafana log panels
Design document	✔ Done	docs/01–03 and this report
OO / functional paradigm	✔ Done	Pydantic models, dataclasses, module functions
LLD with API I/O specs	✔ Done	Chapter 4 of this report
Architecture diagram	✔ Done	Figure 2.1, Chapter 2
HLD diagram	✔ Done	Figure 3.1, Chapter 3
Loose coupling (UI / backend)	✔ Done	Independent Docker services; REST-only interface
Python coding style	✔ Done	`compileall` in CI; type hints throughout
Logging implemented	✔ Done	`configure_logging`; Postgres and Loki
Exception handling	✔ Done	Try/except in every route handler
Inline documentation	◐ Partial	Key modules documented; not all files
Unit test cases	✔ Done	5 unit tests in `tests/unit/`
Test plan	✔ Done	Chapter 6 of this report
Test cases enumerated	✔ Done	Chapter 6, 18 test cases total

Requirement	Status	Evidence
Test report with pass/-fail counts	Pending	Fill `docs/05_TEST_REPORT.md` post dry-run
Acceptance criteria defined	✔ Done	Chapter 6 Section 1
Data ingestion pipeline (Airflow)	✔ Done	Airflow DAG wraps DVC pipeline stages
DVC DAG for CI	✔ Done	`dvc.yaml`; 6-stage DAG
Git and DVC versioning	✔ Done	`.gitattributes`; `dvc.lock`
Experiment tracking (MLflow)	✔ Done	`src/training/train.py`; `mlruns/`
Beyond-autolog tracking	✔ Done	Confusion matrix, per-class F1, duration
Prometheus instrumentation	✔ Done	17 metrics across 3 exporters
Grafana NRT visualisation	✔ Done	Provisioned dashboard JSON
MLflow APIification and MLproject	✔ Done	`MLproject`; `model_loader.py`
FastAPI for UI APIs	✔ Done	`api/app/main.py`
Dockerised backend and frontend	✔ Done	Dockerfiles for API, model service, frontend
Docker Compose separate services	✔ Done	`docker-compose.yml`; 20+ services
User manual	✔ Done	Chapter 7 and `docs/06_USER_MANUAL.md`

Chapter 12

Deployment Runbook

12.1 Environment Variables

```
AIRFLOW_UID=50000
_AIRFLOW_WWW_USER_USERNAME=airflow
_AIRFLOW_WWW_USER_PASSWORD=airflow
SMTP_HOST=smtp.mailtrap.io
SMTP_PORT=587
SMTP_USER=<your_mailtrap_user>
SMTP_PASSWORD=<your_mailtrap_password>
ALERT_EMAIL_FROM=alerts@galaxy.local
ALERT_EMAIL_TO=you@example.com
```

12.2 Starting, Verifying, and Stopping

```
# Start all services
docker compose up -d --build
docker compose ps # verify all services healthy
docker compose logs -f api # tail API gateway logs

# Run DVC pipeline manually inside the trainer container
docker compose exec trainer bash
dvc repro report

# Stop (keep volumes)
docker compose down

# Stop and destroy all volumes (destructive)
docker compose down -v
```

Appendix A

Prometheus Alert Rules

Key alert rules defined in `monitoring/prometheus/alerts.yml`:

- **HighAPIErrorRate:** Fires when the API 5xx error rate exceeds 5% over 5 minutes.
- **ModelServiceDown:** Fires when the model service `/ready` returns 503.
- **LowModelAccuracy:** Fires when `galaxy_pipeline_test_accuracy` falls below 0.70.
- **DataDriftDetected:** Fires when `galaxy_input_brightness_zscore` exceeds 3.0.

Appendix B

Proof of Deployment Checklist

Table B.1: Deployment proof screenshots (save in `image/proof/`)

#	Description	Filename
1	Docker Compose all containers healthy	`01-docker-compose-ps.png`
2	Frontend prediction UI	`02-frontend-ui.png`
3	Airflow DAG grid view	`03-airflow-dag.png`
4	MLflow Experiment and Registry	`04-mlflow.png`
5	Prometheus Targets page	`05-prometheus.png`
6	Grafana Dashboard	`06-grafana-dashboard.png`
7	Adminer SQL query result	`07-adminer-sql.png`
8	Alert email in Mailtrap	`08-mailtrap-alert.png`
9	Generated HTML/Markdown report	`09-latest-report.png`